

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

16. Real-Time Clock

Írta: Kalmár Gábor

Lektorálta: Lágler Gergely

Ebben a tananyagrészben a valós idejű órával (real-time clock-kal) ismerkedünk meg. Láthatjuk majd, mikor és milyen esetekben van rá szükség, példákon keresztül válik világossá a használata, és ábrákkal illusztrált leírások segítik a működési elv megértését. Találkozni fogunk olyan esetekkel is, amikor szükségünk lenne a használatára, viszont nem áll rendelkezésre az adott környezetben, de természetesen nem kell pánikba esni ekkor sem. A real-time clock-ról, vagy ahogy a későbbiekben hivatkozni fogunk rá, RTC-ről előljáróban annyit elmondhatunk, hogy ez egy több szempontból is kifejezetten hasznos periféria.

ÁLTALÁNOS INFORMÁCIÓK

FUNKCIONALITÁS

Kezdjük először az elnevezéssel, amiből kiderül, mit is takar valójában egy RTC, ami magyarul valós idejű órát jelent. Sokan az elnevezés láttára a valós idejű rendszerekkel kapcsolhatják össze gondolatban, de ennek ellenére nincs kapcsolat a két fogalom között. Azért nevezik valós idejű órának, mert a valós, emberi formátumú időt méri, tehát az RTC nem más, mint a hétköznapi értelemben vett idő mérésére szolgáló elektronikus eszköz, amit a legtöbb esetben kiegészít egy naptár funkció, továbbá jellemző még a tartalék tápellátás lehetősége is, ami hasznos kiegészítő, ha a rendszerünk nincs mindig bekapcsolva, illetve amennyiben számítanunk kell a tápellátás időnkénti megszakadására.

Általánosságban elmondható, hogy az RTC a hagyományos időzítő (timer) perifériákkal szemben, sokkal kisebb hibával képes a hosszú távú időmérésre.

FELHASZNÁLÁSI KÖRÖK

Alapvetően minden olyan szituációban szükség van rá, ahol ismernünk kell a pontos időt. Lássunk konkrét példákat:

Időmérés

Legegységelműbb esetben olyanokra gondoljunk, amikor a rendszerünk felhasználói felületén valamilyen formában látjuk a pontos időt. Például: PC, okostelefon, okosóra, digitális ébresztőóra, háztartási gépek (mikró, sütő), TV, set top box, hifi. A sort még hosszan lehetne folytatni.

Előfordulhat azonban az az eset is, amikor az idő csak kiegészítő információként szolgál a fő funkcionalitásban. Természetesen ilyenkor is elengedhetetlen az RTC megléte. Legszenvedelmesebb példa egy fájlrendszer működése, mert itt mentésre kerül a fájlok létrehozási, módosítási, sőt akár

még a hozzáférési ideje is. A másik tipikus ilyen jellegű felhasználási forma a naplózás, ahol szinte kötelező egy-egy bejegyzéshez elmenteni az esemény bekövetkezésének pontos idejét is.

Időzítés

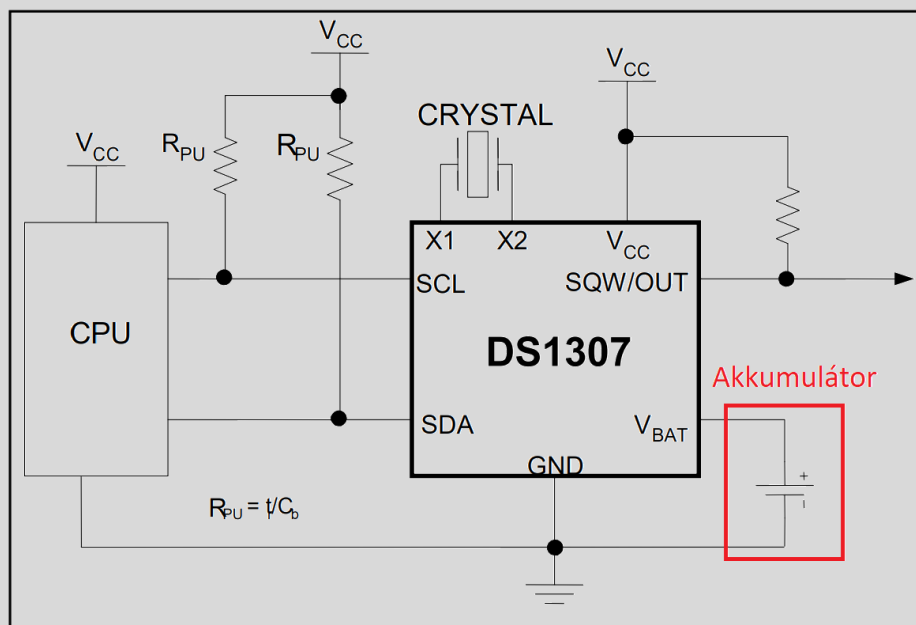
Az RTC-t időzítésekre is lehet használni. Felvehetünk periodikus, egyszeri vagy többszöri időzítéseket is a feladattól függően, amire rengeteg példát lehet felhozni. Gondoljunk csak arra, hogy a háztartásokban a legtöbb háztartási gépnek van időzítő funkciója, legyen szó mosógépről, sütőről, mikróról vagy akár TV-ről. A legkézenfekvőbb példa pedig az ébresztőóra, ahol több ébresztést is be tudunk állítani, akár a szundi időtartamát is testreszabhatjuk, vagy vegyük példának a személyi számítógépeket, ahol a PC RTC-jét például beállíthatjuk a BIOS-ban, így adott időpontban lehet bekapcsolni vagy felébreszteni a számítógépünket. Fontos az időzítés a mérőberendezésekben is, hiszen képzeljünk el egy levegőminőség ellenőrző állomást, aminek a feladata 10 percenkénti mérés és adattovábbítás.

TÍPUSAI

Kitekintés:

Külső hardver egység

Valós idejű órák külső hardveregységként is elérhetőek, amik akkor használatosak, ha a mikrokontrollerünk nem tartalmaz integrált RTC-t. Ebben az esetben egy kommunikációs buszon, pl. SPI-on vagy I2C-n keresztül tudunk kommunikálni egy ilyen IC-vel, ami lehet például a BQ32000 (Texas Instruments), DS1307 (Maxim Integrated), vagy az M41T0 (ST Microelectronics). A felhasznált valós idejű óra igényelhet további komponenseket is, például az alábbi ábrán egy I2C illesztésű RTC látható, amihez külső kristályoszillátort, és tartalék áramforrást is kapcsoltak.



1. ábra: Külső RTC (DS1307) illesztése

Forrás: Maxim Integrated

<https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>

Belső szoftver megvalósítás

RTC-t szoftveresen is meg lehet valósítani. Ezt az utat akkor választják, ha szükség lenne a funkcionalításra, de integrálva nem elérhető, továbbá külső hardver egység használata sem megoldható. Ilyen szituáció akkor állhat elő például, ha egy kész rendszeren utólag kell funkcióbővítést eszközölni, vagy esetleg már a kezdetektől a költségcsökkentés jegyében nem is számoltak fizikai RTC-vel. A szoftveres megvalósítás a mikrovezérlő egyéb, kevésbé pontos időzítő (timer) egységeire támaszkodik. A legegyszerűbb módszer, ha egy időzítőt megszakításos módban használnak, úgy, hogy 1 másodpercenként generáljon megszakítást. Ennek az eseménynek a lekezelésekor növelik egy változó értékét, ami a másodperceket tárolja. Ha ez az érték eléri az 59-et, akkor a következő növelésnél a másodperceket ki kell nullázni, és egy másik változót kell növelni, ami a perceket hivatott tárolni, és így tovább. Hátránya a kisebb pontosság és a tartalék tápellátás lehetőségének a hiánya. Abban az esetben, amikor egy eszköznek van aktív internet kapcsolata, akkor azt is meg lehet oldani, hogy időnként egy időszerverrel szinkronizálják a szoftveres órát a hálózati idő protokollon (Network Time Protocol, NTP) keresztül. Ilyen megoldásra láthatunk példát a Raspberry Pi miniatűr számítógép esetében, de emellett természetesen időszinkronizációra egyéb kommunikációs csatorna segítségével is lehet lehetőségünk. Beágyazott rendszereknél szokás soros kommunikációs buszokat, vagy valamilyen rádiós forrást, például GPS-t vagy DCF77-et használni erre a célra.

Belső hardver egység

A legtöbb modern 32 bites beágyazott vezérlőegység tartalmaz integrált RTC-t. A jelen tananyagban ismertetésre kerülő STM32F405VGT6 esetében is beszélhetünk beépített valós idejű óráról. A továbbiakban ennek az egységnek a bemutatását és használatát taglaljuk.

STM32F4 RTC PERIFÉRIA

HARDVER BEKÖTÉSE

Vegyük használatba az STM32F405VGT6 RTC perifériáját, amihez két dolgot kell előzetesen eldöntenünk. Az első az órajel kiválasztása, ugyanis az RTC-nek szüksége van egy alap órajelre, ami érthető is, hiszen az RTC is egyfajta timer modul, (ezekről pedig olvashattunk már a 15-ös tananyagrészen), aminek adhatunk külső forrásból órajelet egy kristályoszillátorról, illetve használhatunk belső forrást is. A fejlesztőpanelen az előbbi, azaz külső kvarcoszillátor került beépítésre. A másik eldöntendő kérdés a tartalék tápellátás használata, ami egy választható funkció, de érdemes élni vele. A fejlesztőpanelen található ilyen kiegészítés, amit nézzünk is meg, hogyan lett kialakítva.

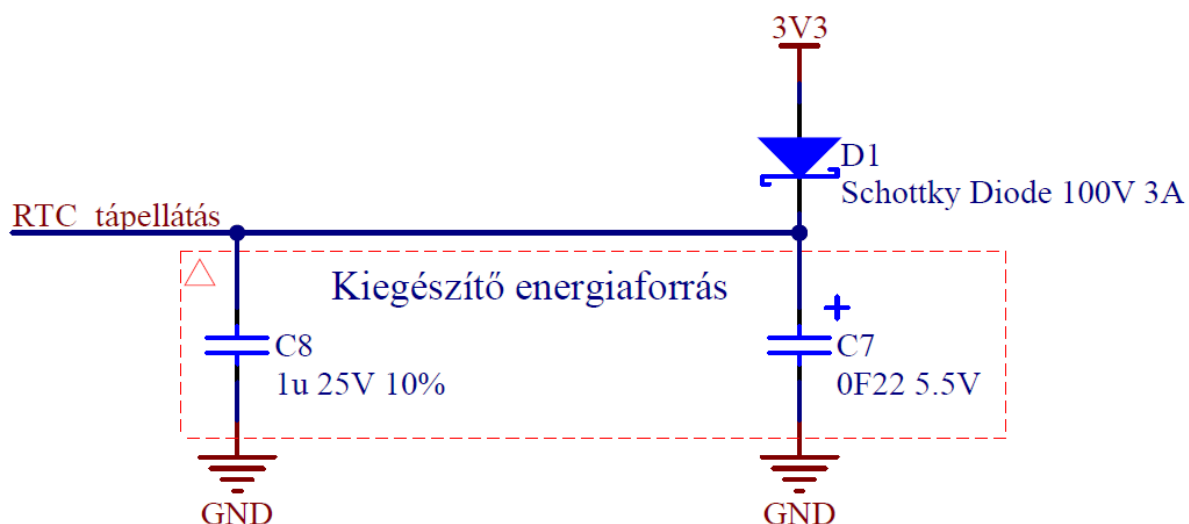
Tartalék tápellátás bekötése

A tartalék tápellátás egy óra esetében jól tud jönni. Segítségével, ha az IC fő tápellátása megszűnik, az RTC továbbra is aktív marad, következésképpen nem "felejt el" az időt a rendszerünk. Az általunk használt kontrollernek van egy VBAT jelű bemenete, amire 1,65 V és 3,6 V közötti energiaforrás kell kapcsolni. Több lehetőség közül választhatunk, például elem, akkumulátor vagy kondenzátor töltheti be ezt a szerepet. Nézzük is meg egyenként ezen lehetőségeket!

Nagyon gyakran a nem tölthető lítium alapú gombelemekkel találkozhatunk, ezen belül is a CR és BR jelölésűekkel, mint tartalék energiaforrás, hiszen magas az energiasűrűségük, és alacsony az önkisülésük.

Tartalék energiaforrásnak akkumulátort is használhatunk többféle kivitelben. A legelterjedtebbek a gombelem nagyságú, ML jelölésű lítium cellák, és a pár tagból soros kapcsolással összeállított NiCd vagy NiMH akkuk, amiknek közös jellemzője, hogy töltőáramkör használata szükséges, ez pedig a további költséget jelent. Hátrány lehet a töltési ciklusok limitált száma, azaz számolnunk kell az akkuk öregedésével, kiváltképp, ha sokszor kell áramkimaradásra számítani. Használatuknak abban az esetben lehet előnye, amennyiben az RTC mellet más áramköri elemet is szeretnénk szünetmentes energiaforrással ellátni, mert ebben az esetben jól jöhet a többletkapacitás, amit akkumulátorokkal könnyebben tudunk biztosítani, így áthidalva akár hosszabb időintervallumokat is.

Végző soron használhatunk kondenzátort is, amelynek esetében a működés gondozásmentes, de azért egy viszonylag egyszerű töltő áramkörre szükség van. Az elemhez és az akkumulátorhoz képest csak erősen limitáltabb időtartamú áramszüneteket tudunk kezelni, de cserébe könnyű beépíteni és kevesebb szabályozás vonatkozik az ezekben felhasznált kémiai anyagok szállítására és megsemmisítésére. A fejlesztőpanelen kondenzátor alapú megoldást találunk, aminek a bekötése az alábbi ábrán szemléltetett módon történt:

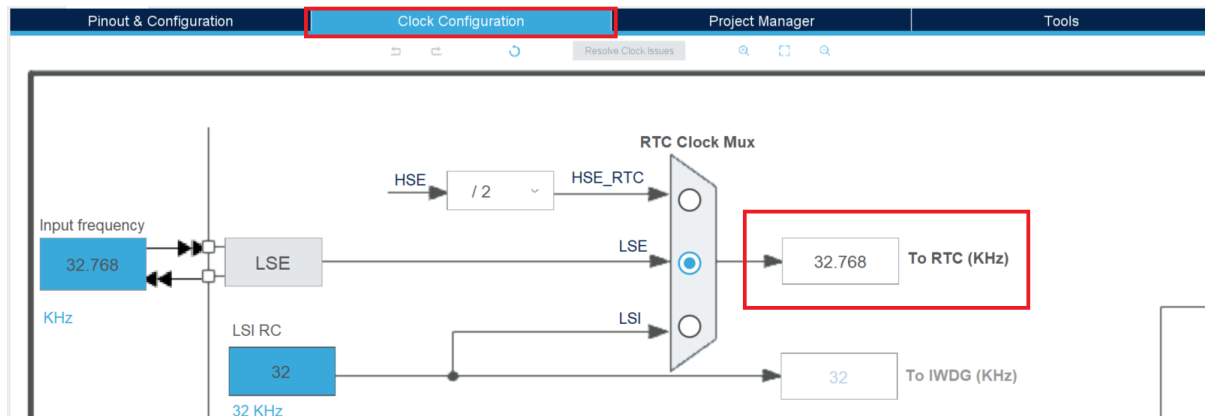


2. ábra: Kondenzátor bekötése

A VBAT csak bemenete a mikrokontrollernek, ezért nem tudja tölteni a kondenzátort, így a töltésről a fő tápvonal felől kell gondoskodnunk. Ha egyszerűen csak rákötnénk a kondenzátorra az elsődleges áramforrást, akkor áramkimaradás esetén a kondenzátor az RTC periféria mellett más áramköri elemeket is ellátna árammal, következésképpen nagyon gyorsan lemerülne. Ennek kivédésére egy diódát kell beiktatni, ami üzemszerűen csak egy irányba engedni fogja az áramot, így, amikor él a fő tápvonal, akkor a diódán keresztül kap a VBAT energiaellátást, és a kondenzátor is töltődik, de amennyiben megszűnik a fő tápellátás, akkor a kondenzátorban tárolt elektromos töltés látja el a VBAT bemenetet árammal, a dióda pedig megakadályozza, hogy a fő tápvonalon keresztül ez a töltés gyorsan elillanjon.

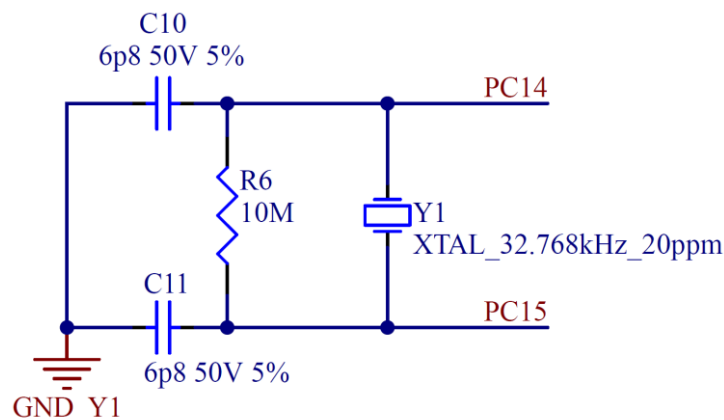
Kristályoszillátor illesztése

Összesen 3 féle módon tudunk órajelet biztosítani az RTC perifériának. Lehetőség van belső 32 kHz-es RC oszillátor (LSI), külső nagysebességű oszillátor (HSE), illetve külső alacsony sebességű oszillátor (LSE) alkalmazására. Ez utóbbi megoldás található a fejlesztőpanelen, ahol az oszillátor frekvenciája 32,768 kHz, ami kettőnek a 15. hatványa. Ez azért fontos, mert ebből nagyon pontosan elő lehet állítani az idő alapegységét, az 1 másodpercet, amit úgy tehet meg a mikrokontroller, hogy az induló frekvenciát 15-ször elosztva 2-vel, végül 1 Hz-et kap. Szemléletesen az RTC órajel beállítása a következőképpen alakul:



3. ábra: RTC modul órajel beállítása

A kristályoszillátort a mikrovezérlő 8-as (bemenet) és 9-es (kimenet) lábára kell kötni, amelyek a "C" jelölésű port 14-es és 15-ös tagjai, így a kapcsolási rajzon PC14 és PC15 nevekkkel hivatkozunk rájuk. Lássuk a végső illesztő kapcsolást:



4. ábra: Kristályoszillátor bekötése

C10 és C11 feladata a frekvencia stabilitásának a biztosítása, amiknek az elméleti értékét a kvarc gyártója adja meg. A pontosság nagymértékben függ a felhasznált alkatrészek minőségétől is. A kondenzátorok esetében érdemes első osztályú kerámia kondenzátorokat használni, melyeknek a lehető legkisebb a hőmérsékletfüggése, amiket NPO-val vagy COG-val szoktak jelölni. A kvarckristály esetében is fontos a pontosság, amit ppm/°C-ben mérnek, ahol a ppm az angol parts per million rövidítése. 1 ppm/°C azt jelenti, hogy 1 °C hőmérsékletváltozás hatására a frekvencia 1 milliommód részével változik meg a 25 °C referencia hőmérséklethez megadott értékhez képest. További információk a kristályokról és a bekötésükről a mikrokontroller adatlapjában a 5.3.8-as fejezetben, illetve az AN2867 alkalmazási segédletben található.

HASZNÁLAT

Az STM32 fejlesztői ökoszisztémájában kétféleképpen használhatjuk a perifériákat. Az egyik elérési mód az alacsony szintű (LL) alkalmazásprogramozási felület, a másik pedig egy magasabb szintű hardverleíró réteg (HAL). A kétféle API között a Project Manager/Advanced Settings menüpontban tudunk választani az STM32CubeIDE-ben. Az ötödik tananyagrészben megismerkedtünk a HAL függvénykönyvtárral, illetve az eddigi példák is erre épültek, ezért az RTC használatát itt is a HAL driverek segítségével mutatjuk be.

Beállítások

A konkrét függvényhívások ismertetése előtt nézzük meg, hogyan kell konfigurálni az RTC perifériát, amihez importáljuk be a tananyagrészhez tartozó projektet. A CubeMX nézetben a valós idejű óra beállításait a timerek között fogjuk találni.

Nézzük is meg miket tudnánk beállítani!

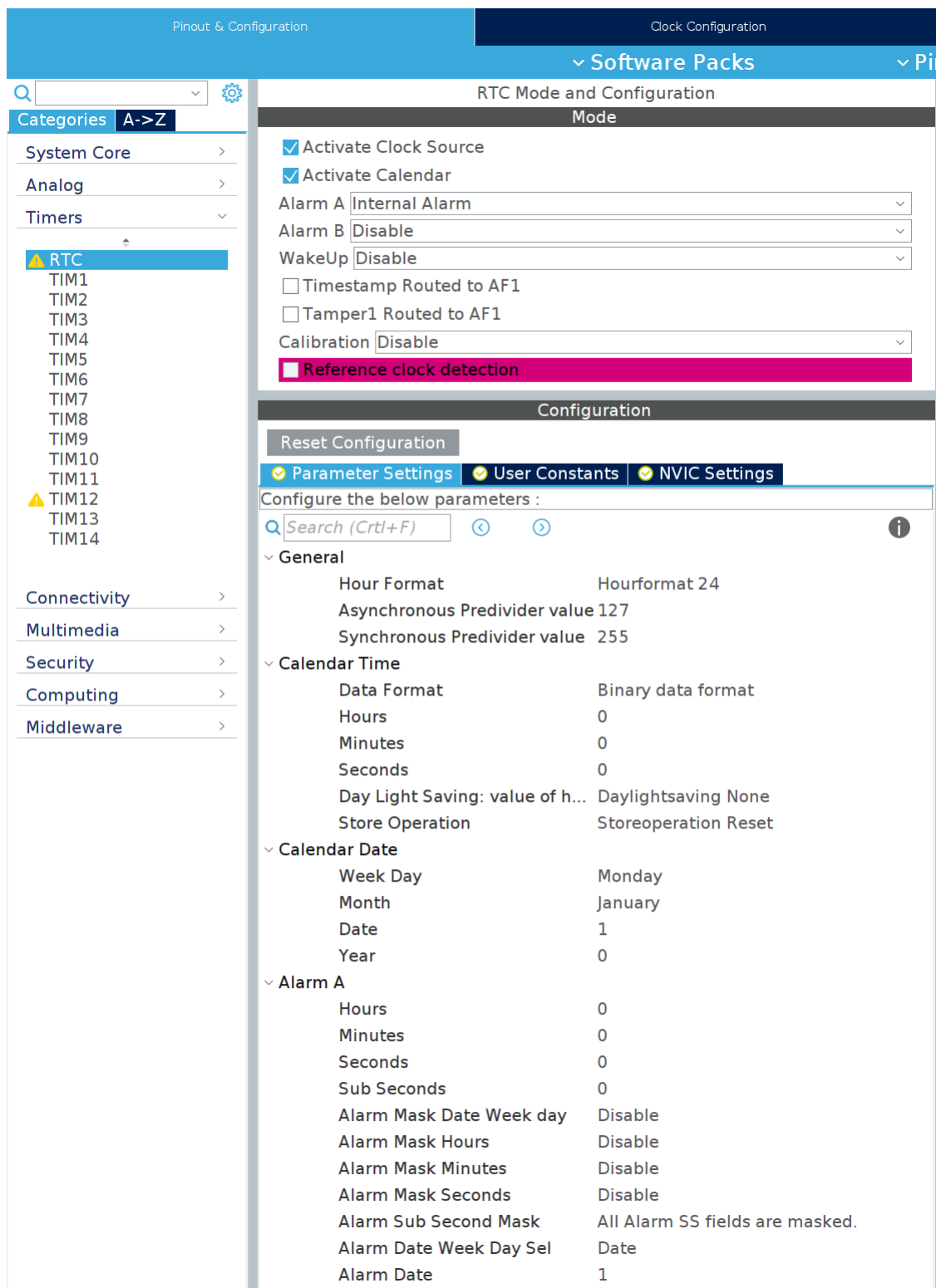
Az Activate Clock Source jelölőnégyzet segítségével válik aktívvá az RTC, ami a gyakorlatban azt jelenti, hogy a CubeMX innentől kezdve generál kódot az RTC-hez. A mentéskor generálásra kerülő kódok a 4. fejezetben elmondottaknak megfelelően alapértelmezetten egy fájlba kerülnek, amely nem a legegyszerűbb megoldás, ezért érdemes elvégezni az ott ismertetett beállításokat.

A következő beállítás az Activate Calendar, ami, ha ki van jelölve, akkor indulásnál egy előre beállított érték kerül a naptárba és az órába, amiket a Parameter Settings tab alatt, a Calendar Time, és a Calendar Date lenyíló részekben tudunk beállítani.

Ha szeretnénk riasztást beállítani, úgy az "Alarm A", vagy "Alarm B" legördülő menüből kell kiválasztanunk, vagy az Internal Alarm-ot, vagy a Routed to AF1-et. Az "A" és a "B" két külön csatornát jelöl, ezek egymástól függetlenül képesek működni. A két opció közül az internal belső riasztást jelent, a routed pedig azt jelöli, hogy ki lett vezetve külső lábra, jelen esetben az RTC_AF1-re. (Az adatlap és a CubeMX elnevezései között sajnos néha különbség van, ugyanis az előbbi "RTC_OUT" néven hivatkozik ugyanerre a lábra.) A tananyagban használt vezérlőegység esetében ez a láb sorrendben a hetedik, és a "C" port 13-as tagja (PC13). Ha megszakításos módban szeretnénk használni a riasztást, akkor azt külön engedélyezni kell az NVIC Setting fül alatt. A megszakításos mód azt jelenti, hogy nem kell folyamatosan lekérdezzetnünk a riasztás állapotát, mert a beállított időpontban a HAL_RTC_AlarmIRQHandler() megszakításkezelő függvény hívódik meg, felfüggesztve az aktuálisan végrehajtott feladatot. Önmagában csak a megszakításhoz kapcsolódó regisztereket kezeli, viszont meghívja a HAL_RTC_AlarmAEventCallback() vagy a HAL_RTCEx_AlarmBEventCallback() függvények valamelyikét a csatornától függően, amiknek a tartalma tetszőleges, megírásuk a felhasználóra van bízva. Alapvetően arra jó a megszakításos mód, hogy értesítést kapjunk a riasztásról.

A WakeUp funkció hasonló a riasztáshoz, de jobban hasonlít egy klasszikus időzítőhöz. Segítségével adott idő elteltével fel lehet ébreszteni a kontrollert, az időt viszont a riasztással ellentétben nem órában, percben és másodpercben kell megadnunk, hanem egy számlálót kell definiálnunk, úgy, hogy meg kell adni egy értéket és egy időalapot. Az érték egy 32 bites szám lehet, amely definiálja, hogy mikor történjen a felébresztés, az időalap pedig a "lépések" hossza, amely az RTC periféria alapórjelének leosztása, azaz 16, 8, 4, 2, vagy 1 Hz lehet.

Találhatunk még beállításokat például az időbélyegre és a kalibrációra vonatkozóan, amiket nem taglalunk ennek a tananyagnak a keretei között, de a kíváncsi tanulók az STMicroelectronics weboldaláról letölthető AN3371 és az AN4759 alkalmazási segédletben további információkat találhatnak.



The screenshot displays the STM32CubeMX configuration tool. On the left, the 'Categories' list includes System Core, Analog, Timers, RTC (highlighted), and various TIMx modules. The main panel is titled 'RTC Mode and Configuration'. It features a 'Mode' section with checkboxes for 'Activate Clock Source' and 'Activate Calendar', and dropdown menus for 'Alarm A' (Internal Alarm), 'Alarm B' (Disable), 'WakeUp' (Disable), and 'Calibration' (Disable). A 'Reference clock detection' option is also present. Below this is the 'Configuration' section, which includes a 'Reset Configuration' button and tabs for 'Parameter Settings', 'User Constants', and 'NVIC Settings'. The 'Parameter Settings' tab is active, showing a search bar and a list of parameters organized into sections: General (Hour Format, Asynchronous Predivider value, Synchronous Predivider value), Calendar Time (Data Format, Hours, Minutes, Seconds, Day Light Saving, Store Operation), Calendar Date (Week Day, Month, Date, Year), and Alarm A (Hours, Minutes, Seconds, Sub Seconds, Alarm Mask Date Week day, Alarm Mask Hours, Alarm Mask Minutes, Alarm Mask Seconds, Alarm Sub Second Mask, Alarm Date Week Day Sel, Alarm Date).

5. ábra: RTC periféria a CubeMX-ben

HAL függvények

Nézzük meg milyen könyvtári függvényeken keresztül tudjuk elérni az egyes funkciókat!

HAL_RTC_Init()

Függvény funkciója	Felkészíti és alapállapotba helyezi az RTC perifériát. A CubeMX gondoskodik a helyes meghívásáról, nekünk nincs vele dolgunk.
Paraméterek	• hrtc: Mutató az RTC beállításaira.

HAL_RTC_GetTime()

Függvény funkciója	Lekéri az időt.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • sTime: Mutató egy idő struktúrára. • Format: A formátumot adja meg, ami lehet RTC_FORMAT_BIN vagy RTC_FORMAT_BCD. A BIN binárisat jelent, a "BCD" pedig binárisan kódolt decimális számot, ahol 2 decimális számjegyet tárolunk egy bájtban. Szemléltetésképpen a dél, mint tizenkettedik óra, így néz ki hexadecimálisan (1 számjegy 4 bit) leírva: 0xC Így pedig BCD esetében: 0x12

HAL_RTC_GetDate()

Függvény funkciója	Lekéri a dátumot.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • sDate: Mutató egy dátum struktúrára. • Format: A formátumot adja meg. Lehet RTC_FORMAT_BIN vagy RTC_FORMAT_BCD.
Megjegyzés	Ezt a függvényt csak közvetlenül a HAL_RTC_GetTime() után hívhatjuk meg. Ez azért van, mert a HAL_RTC_GetTime() meghívása blokkolja a dátumhoz tartozó ideiglenes másolatot képző regisztereket és a HAL_RTC_GetDate() oldja fel ezeket, amire azért van szükség, hogy az idő és a dátum összetartozó értékpárt tudjon alkotni. A lényeg, hogy mindig párban kell lekérni az időt és a dátumot a sorrend betartásával. (Ezt a szakmában szivárgó absztrakciónak hívjuk, ami egyfajta hiba a magasabb szintű API-kban. A dátum és idő lekérdező függvényeknek teljesen el kellene fedniük a regiszterek szintjét előlünk, de mégsem ez történik.)

HAL_RTC_GetAlarm()

Függvény funkciója	Lekérdezi a beállított riasztás idejét.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • sAlarm: Mutató egy riasztás struktúrára. • Alarm: Megadhatjuk melyik csatornát kérjük le a kettő közül. Értéke lehet RTC_ALARM_A vagy RTC_ALARM_B. • Format: A formátumot adja meg. Lehet RTC_FORMAT_BIN vagy RTC_FORMAT_BCD.

HAL_RTC_SetTime()

Függvény funkciója	Beállítja az időt.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • sTime: Mutató egy idő struktúrára. • Format: A formátumot adja meg. Lehet RTC_FORMAT_BIN vagy RTC_FORMAT_BCD.

HAL_RTC_SetDate()

Függvény funkciója	Beállítja a dátumot.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • sDate: Mutató egy dátum struktúrára. • Format: A formátumot adja meg. Lehet RTC_FORMAT_BIN vagy RTC_FORMAT_BCD.

HAL_RTC_SetAlarm_IT()

Függvény funkciója	Beállítja a riasztást megszakításos módban.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • sAlarm: Mutató egy riasztás struktúrára. • Format: A formátumot adja meg. Lehet RTC_FORMAT_BIN vagy RTC_FORMAT_BCD.

HAL_RTC_DeactivateAlarm()

Függvény funkciója	Kikapcsolja a riasztást.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • Alarm: Megadhatjuk melyik csatornát kérjük le a kettő közül. Értéke lehet RTC_ALARM_A vagy RTC_ALARM_B.

HAL_RTC_AlarmIRQHandler()

Függvény funkciója	Ez a függvény kezeli a riasztás megszakítás kérését. Amikor eljön a riasztás ideje, a vezérlés átugrik erre a függvényre, és az aktuálisan végrehajtott egyéb művelet megszakad . A függvény visszatérése után a vezérlés visszakérül a megszakított kódrészre.
Paraméterek	• hrtc: Mutató az RTC beállításaira.

HAL_RTC_AlarmAEventCallback()

Függvény funkciója	Az "A" riasztási csatorna megszakítás esetén visszahívott függvénye. Ezt a függvényt a felhasználónak kell definiálnia. A tartalma tetszőleges.
Paraméterek	• hrtc: Mutató az RTC beállításaira.

HAL_RTCEX_BKUPRead()

Függvény funkciója	Az RTC-nek van 20 db tartalék regisztere is. Ezekbe tetszőleges információt lehet írni és megőrizik a tartalmukat addig, amíg van tartalék tápellátás biztosítva. Ez a függvény ki tud olvasni egy ilyen regisztert.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • BackupRegister: A megcímzett regiszter sorszáma 0 és 19 között.

HAL_RTCEX_BKUPWrite()

Függvény funkciója	Ez a függvény annyiban különbözik a HAL_RTCEX_BKUPRead()-tól, hogy nem olvassa, hanem írja az adott regisztert.
Paraméterek	• hrtc: Mutató az RTC beállításaira. • BackupRegister: A megcímzett regiszter sorszáma 0 és 19 között. • Data: Adat, amit beírunk a regiszterbe.

Természetesen nem csak ennyi függvénye van az RTC modul API-jának. Ezeket azért mutattuk be, mert ezek a leggyakrabban használtak, és a későbbiekben előkerülnek a példa alkalmazás kapcsán is. Az API-k teljes listáját és leírását az interneten elérhető UM1725-ös felhasználói dokumentációban találhatjuk.

PÉLDA ALKALMAZÁS

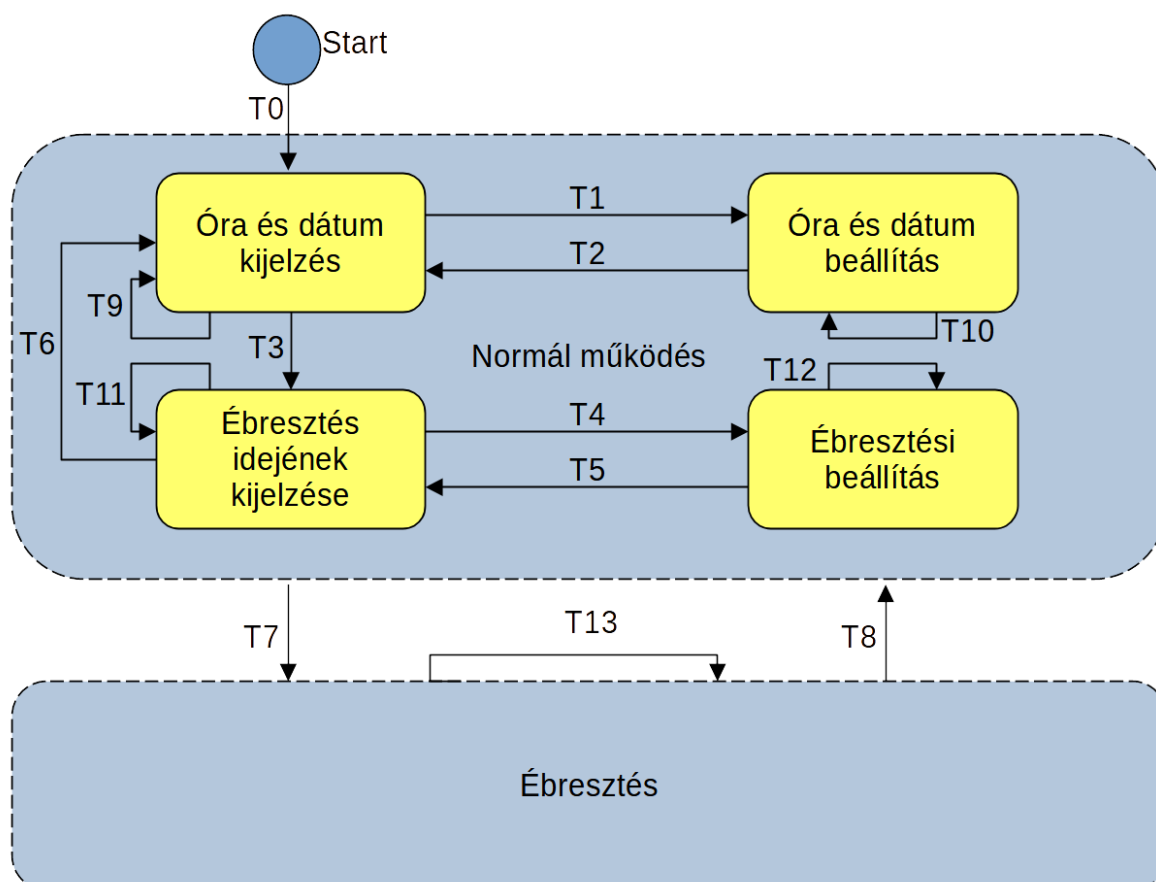
A működés egy ébresztőóra példáján keresztül kerül bemutatásra. Ebben az alfejezetben a fejlesztés lépéseit mutatjuk be, és a kód működését magyarázzuk el.

Leírás

Első lépésként az alkalmazás működését kell meghatározni, ehhez szedjük pontokba milyen funkciói vannak egy egyszerű ébresztőórának:

- Pontos idő és dátum kijelzése.
- Pontos idő beállítása.
- Ébresztés idejének kijelzése.
- Ébresztés beállítása.
- Ébresztés.

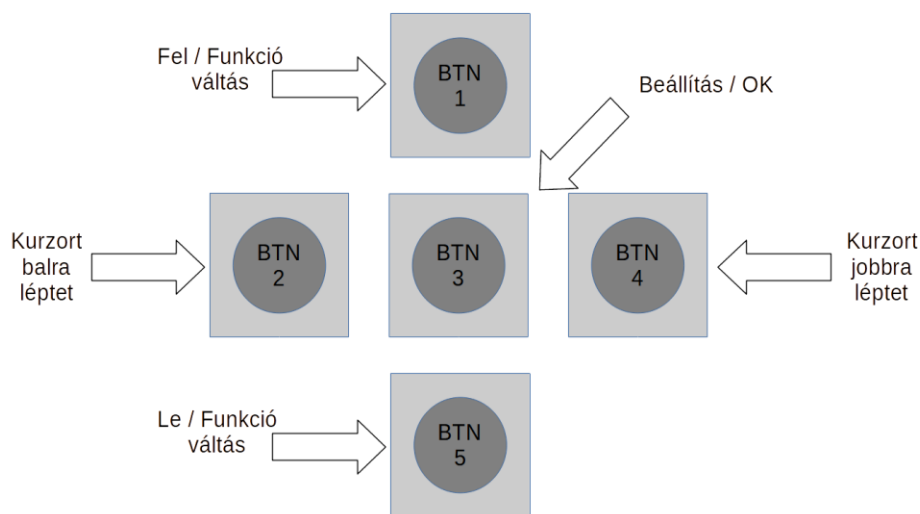
A beállításhoz a gombokat fogjuk használni, míg a kijelzés és az ébresztés a fejlesztőpanelhez csatlakoztatható OLED kijelzőn történik, ahol az ébresztés természetesen csak demonstrációs célt szolgál képernyő villogtatással. A funkcionalitás birtokában a következő lépés a működés modellezése és a szoftver felépítésének meghatározása. A sikeresen definiált üzemállapotok egy állapotgépbe szervezhetők, amiben az állapot az, ahol a rendszer tartózkodik, egyes események hatására egyik állapotból átkerülhet egy másikba, amiben bizonyos tevékenységeket hajt végre. Ennek a legszemléletesebb grafikus leírása az állapotgráf, amiben a téglalapok állapotokat jelölnek, a nyilak pedig átmeneteket, amiknek a leírását fel lehet venni egyből az ábrába, illetve táblázatos formában is. Ehhez a példához az utóbbit fogjuk használni, mert a tervezésnek ebben a korai szakaszában a pontos események és tevékenységek még nem ismertek, viszont az állapotokat és a lehetséges átmeneteket viszont már felvehetjük:



6. ábra: Ébresztőóra állapotleírása

Feltűnhet, hogy az állapotok nem egy szinten lettek meghatározva, aminek az az oka, hogy az ébresztés egy speciális állapot, hiszen bármelyik másik állapotból el tudunk jutni ide. Ha egy szinten határoztuk volna meg az állapotokat, akkor lényegesen bonyolultabb gráfot kaptunk volna, ennek elkerülése érdekében az összetartozó státuszokat csoportosítottuk oly módon, hogy a kék téglalapok jelölik a főállapotokat, a sárgák pedig az alállapotokat.

A következő lépés az átmeneteket kiváltó pontos események és az átmenet során lezajló tevékenységek meghatározása, amihez ki kell találni a gombok pontos szerepét. Nézzük is meg mit csinálnak! Normál működés során az óra és dátum kijelzése és az ébresztés idejének kijelzése között a fel (BTN1), és a le (BTN5) gombokkal lépkedhetünk, a beállításhoz a középső gombot kell megnyomni. Értelmszerűen, ha éppen az óra és a dátum van kijelvezve, akkor a középső (BTN3) gomb ennek a beállítására ugrik, ahol már a balra (BTN2) és jobbra (BTN4) gombokkal lehet a kijelölést léptetni. A fel és a le gombok a kijelölt érték növelésére vagy csökkentésére szolgálnak, a középsővel pedig a szerkesztést lehet befejezni. Az ébresztésből a középső gomb megnyomásával tudunk kilépni.



7. ábra: Gombok szerepe és fizikai elhelyezkedése

Ez az a pont, ahol a felépítés már világos és kiegészíthetjük az állapotgráfot az átmenetek részletes leírásával. Jöjjön hát a táblázat, ami leírja a végrehajtandó műveleteket az állapotátmenetek során, amelynek oszlopai a következőket tartalmazzák:

- állapotátmenet: az átmenet azonosítója (a T az angol transition rövidítése),
- feltétel: olyan esemény, aminek a hatására végbemegy az átmenet,
- tevékenység: az átmenet hatására lefutó parancs(ok).

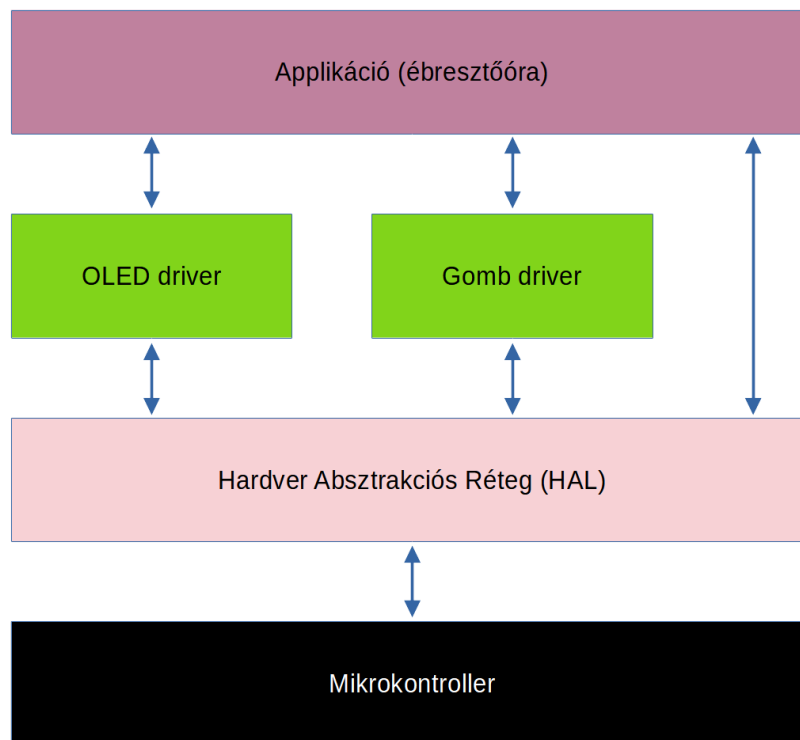
Állapotátmenet	Feltétel	Tevékenység
T0	Sikeres hardver inicializáció	A rendszer alapállapotba kerül.
T1	Középső gombot megnyomták.	Pontos idő és dátum lekérése az RTC-től.

Állapotátmenet	Feltétel	Tevékenység
T2	Középső gombot megnyomták.	Pontos idő és dátum beállítása az RTC perifériában.
T3	Fel vagy le gombot megnyomták.	Ébresztés aktuális idejének és állapotának lekérése az RTC-től.
T4	Középső gombot megnyomták.	Ébresztés aktuális idejének és állapotának lekérése az RTC-től.
T5	Középső gombot megnyomták.	Ébresztés idejének és állapotának beállítása az RTC perifériában.
T6	Fel vagy le gombot megnyomták.	Pontos idő és dátum lekérése az RTC-től.
T7	Eljött az ébresztés ideje.	Ébresztés megkezdése.
T8	Középső gombot megnyomták.	Ébresztés deaktiválása.
T9	Nem nyomták meg a középső vagy a fel/le gombok valamelyikét.	Óra és dátum kijelzésének folytatása.
T10	Nem nyomták meg a középső gombot.	Óra beállításának folytatása.
T11	Nem nyomták meg a középső vagy a fel/le gombok valamelyikét.	Ébresztés idejének a kijelzését folytatjuk.
T12	Nem nyomták meg a középső gombot.	Ébresztés beállítását folytatjuk.
T13	Nem nyomták meg a középső gombot.	Folytatjuk az ébresztést.
T13	Nem nyomták meg a középső gombot.	Folytatjuk az ébresztést.

1. táblázat: Állapotátmenetekhez tartozó események

Kód felépítése

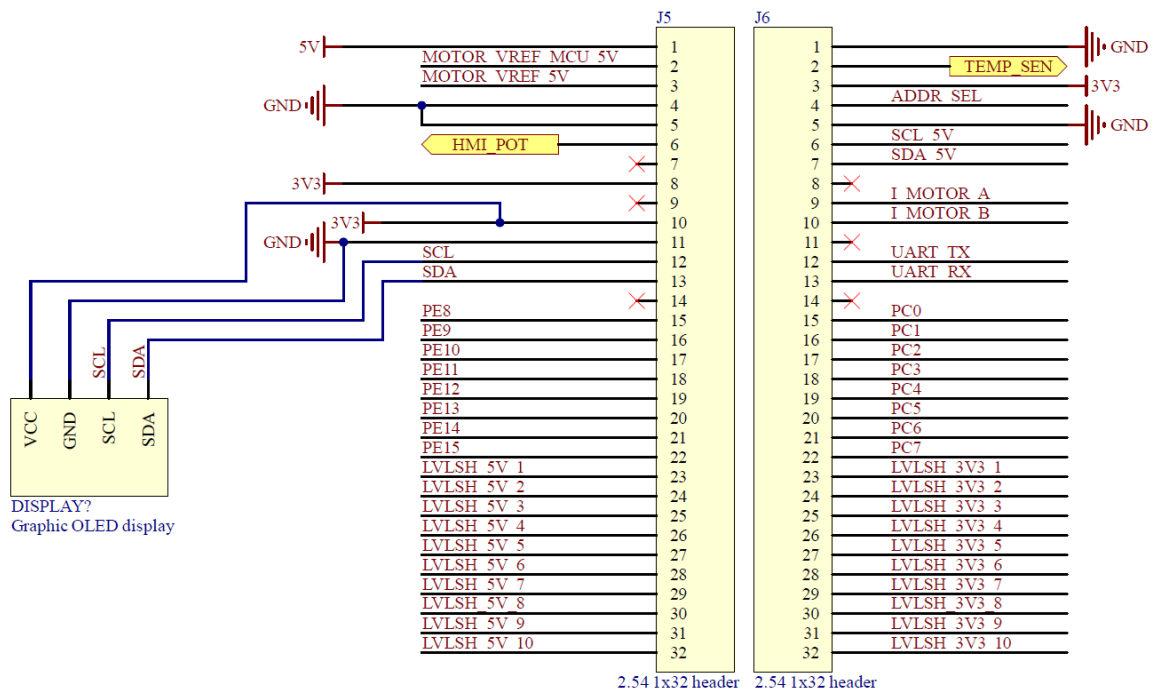
Lassan eljön az ideje a kód ismertetésének, amit az eddigi leírások alapján készítettem el. **A jó kód ismérve az átláthatóság és a karbantarthatóság, mivel egyszer írják, viszont sokszor olvassák!** A fejlesztési folyamat során egy nagyobb problémát kell lebontani kisebb kezelhető részekre, amit az állapotgráffal és az események pontos meghatározásával eddig is teljesítettünk, viszont a kóddal is így kell eljárni: Apróbb, jól körülhatárolható modulokra kell tagolni, amit úgynevezett réteges szoftver felépítéssel célszerű megoldani, ahol legalul a mikrokontroller helyezkedik el, felette áll a HAL függvénykönyvtár réteg, ami felett pedig lehetnek különböző eszközillesztő komponensek, jelen esetben az OLED kijelzőt, és a gombokat kezelő driverek. (Ezek közül az előbbivel már találkozhattunk a korábbi tananyagrészekben, viszont az utóbbit csak itt vezetjük be.) A felépítés tetején az applikáció, azaz az ébresztőóra áll, ami szemléletesen, ábra formájában a következőképpen alakul:



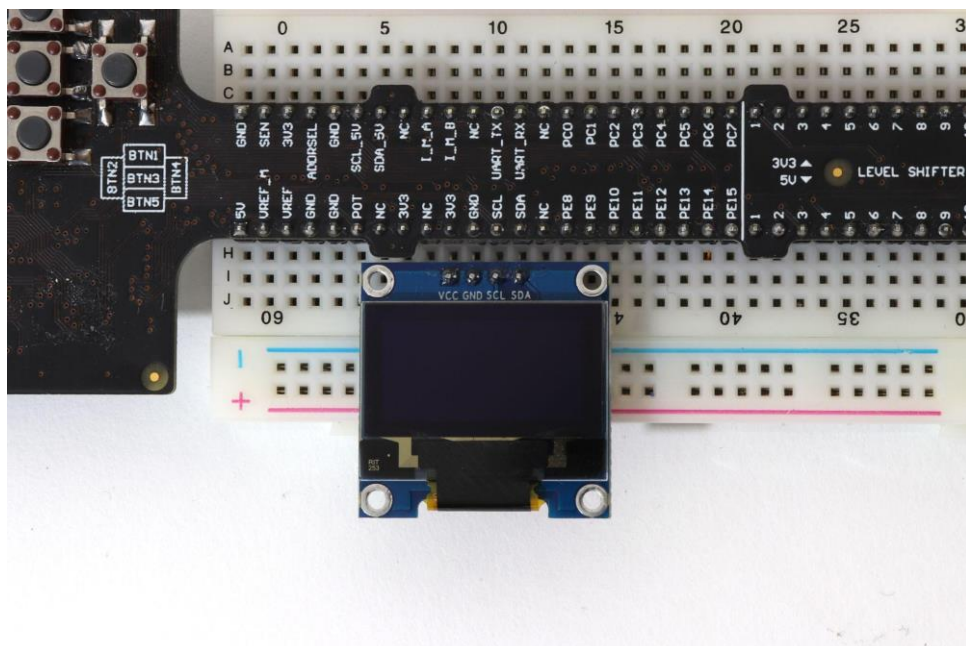
8. ábra: Ébresztőóra réteges felépítése

Kapcsolási rajz

A fejlesztőkártyához csatlakoztatható egy OLED kijelző, amit bekötni az alább szemléltetett módon kell:



9. ábra: OLED kijelző csatlakoztatása - kapcsolási rajz



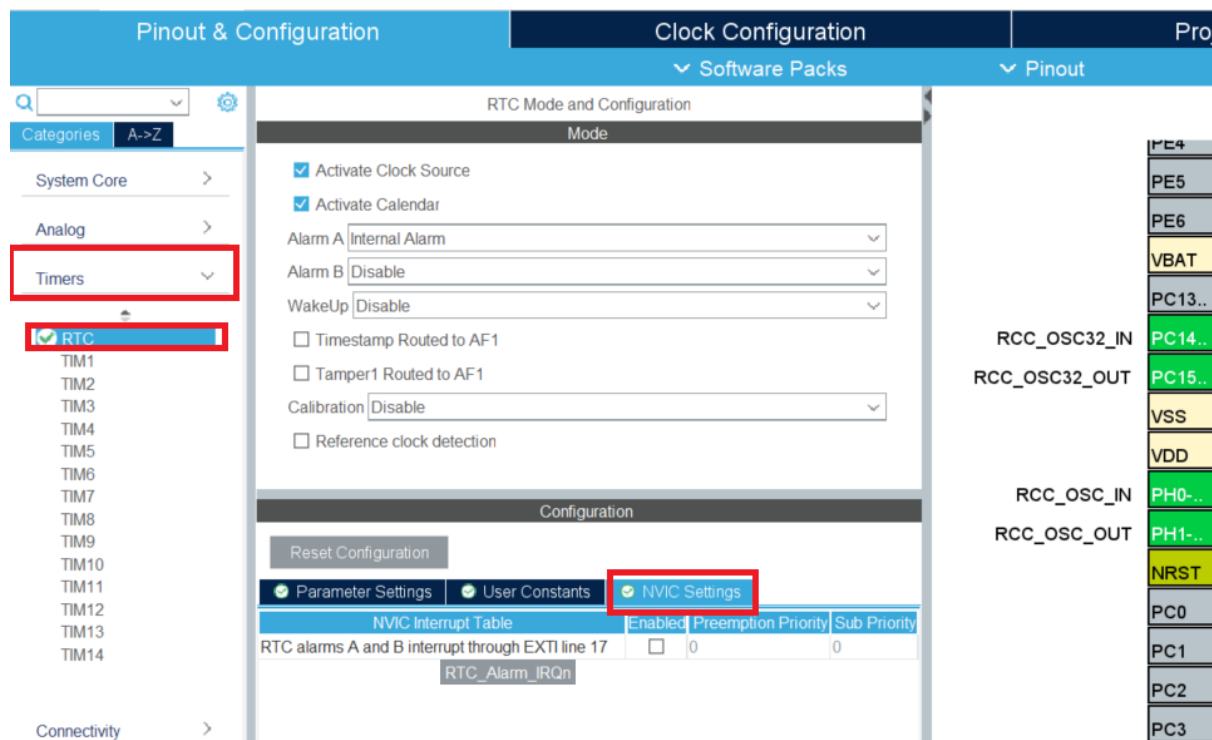
10. ábra: OLED kijelző csatlakoztatása - breadboard

Projekt beállítások

A fejezethez tartozik egy letölthető, kész STM32CubeIDE projekt, amiben megtalálható a teljes forráskód is. Az eddigi tananyagrészekhez képest újdonság lesz az RTC beállítása, illetve a gombokat most egy kicsit másként kezeljük.

RTC

Az RTC beállításai ehhez a konkrét példához úgy alakulnak, hogy használjuk egyrészt magát a valós idejű órát, amit az Activate Clock Source-szal kapcsolunk be, továbbá a naptárat, amit az Activate Calendar-ral, valamint az "A" csatornára kell engedélyezni a riasztást az alábbi képen látható módon:

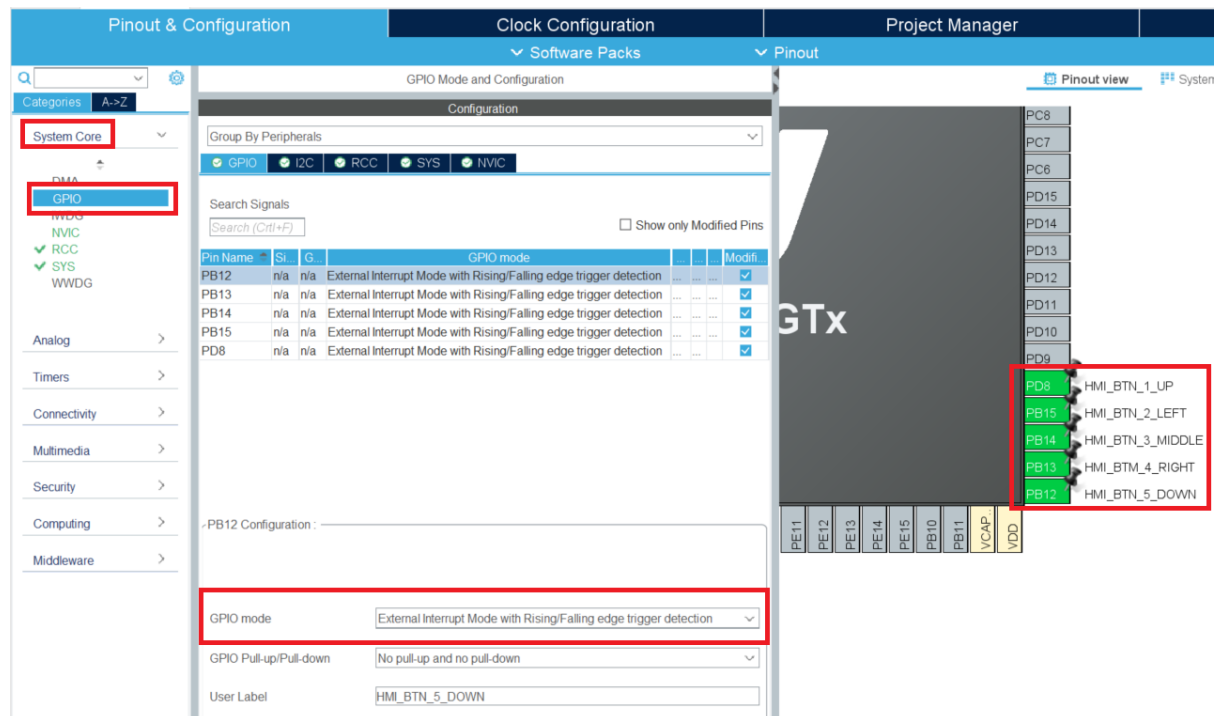


11. ábra: RTC beállításai

Az órajeleket is be kell állítani a Clock Configuration fül alatt, amihez a mellékelt ábra megtalálható a kristályoszillátor illesztése nevű korábbi fejezet részben.

Gombok

Gombokat ez a példaalkalmazás is használ, de az eddigiektől eltérő módon a GPIO mode-ot External Interrupt Mode with Rising/Falling edge trigger detection-re kell állítani, aminek az oka egy későbbi, a gomb driver nevű alfejezetből derül majd ki. A beállítások az alábbi képen láthatóak:



12. ábra: Gombok beállításai

Generált kód

A CubeMX grafikus felületén végzett beállítások C kód formájában kerülnek legenerálásra.

A valós idejű órához tartozó programsorok az rtc.h és rtc.c fájlokba kerülnek.

rtc.h

Ez a fájl csak 2 darab leglényegesebb kódsor:

```
// RTC-hez szükséges változó deklarálása
extern RTC_HandleTypeDef hrtc;

// RTC inicializáló függvény deklaráció
void MX_RTC_Init(void);
```

Az első sor egy változó bejelentése (deklaráció) az RTC-t leíró egy példányban létező (singleton) adatstruktúrára. A "HAL függvények" alfejezetben láthattuk, hogy minden függvénynek van egy hrtc bemenő paramétere, jelen esetben ez az.

A második sor egy függvény publikálása (deklaráció), aminek a feladata a valós idejű óra előkészítése és beállítása (inicializálás).

rtc.c

Az rtc.c fájl tartalmazza egyrészt az rtc.h által bejelentett hrtc struktúra példányosítását, valamint az MX_RTC_Init() függvény megvalósítását. A kód alapvetően beállítja az idő- és dátumformátumot,

valamint engedélyezi az "A" csatornán a riasztásokat megszakításos módban. A programsorokban van egy kézzel írt rész is a USER CODE BEGIN Check_RTC_BKUP és a USER CODE END Check_RTC_BKUP kommentek között. Itt annyi történik, hogy induláskor megvizsgáljuk az egyik tartalék regisztert, aminek a tartalma nem veszik el áramkimaradás alkalmával addig, amíg a tartalék tápellátást szolgáltató kondenzátorban van töltés. Azt a problémát akarjuk kiküszöbölni itt, hogy ne állítsuk alaphelyzetbe minden induláskor az időt és a dátumot. Ha a regiszterben egy előre beállított értéket (0x5050) találunk, akkor biztosak lehetünk benne, hogy előzőleg induláskor már lefutott az idő- és dátumbeállítás, ezért ebben az esetben nem kell ezeket alapértékekre állítani. Abban az esetben viszont, amikor az érték ettől eltérő, azaz az RTC memóriatartalma sérült egy olyan áramkimaradás miatt, amely során az említett kondenzátor is lemerült már, akkor abból az következik, hogy "elfelejtette" a rendszer az időt is, és alapállapotba kell helyezni azt, illetve be kell írni a tartalék regiszterbe a megfelelő értéket (0x5050). A fájlban található továbbá két segédfüggvényt is: HAL_RTC_MspInit(), HAL_RTC_MspDeInit(), amikből az előbbi az RTC perifériát és a riasztáshoz tartozó megszakításokat engedélyezi, utóbbi pedig pont ennek az ellenkezőjét hajtja végre.

Gomb driver

A gombok illesztőprogramja a gombok állapotát, és ezek változását hivatott beolvasni.

buttons.h

Gombonként 2 információra van szükség:

1. Mi a gomb jelenlegi állapota? Ez lehet benyomva (BUTTON_PRESSED), vagy felengedve (BUTTON_RELEASED). Az állapotoknak a ButtonState_TypeDef enum ad nevet és a Button_TypeDef struktúra tárolja le a state tagváltozóban.
2. Volt-e állapotváltozás az előző lekéréshez képest, azaz éppen most nyomták-e meg vagy engedték-e fel a gombot? Ezt az eseményt szintén a Button_TypeDef tárolja, de egy másik változóban, aminek a neve updated, ami két értéket vehet fel: true (igaz) és false (hamis).

Az összes gombot a Buttons_TypeDef struktúra foglalja magában, amit lekérdezni a Buttons_GetStates() függvénnyel lehetséges.

```
#ifndef INC_BUTTONS_H_
#define INC_BUTTONS_H_

#include <stdbool.h>
//Felsorolás a gombok állapotához
typedef enum
{
    BUTTON_RELEASED = 0,    // Felengedett
    BUTTON_PRESSED        // Benyomott
} ButtonState_TypeDef;

// Állapotváltozás detektáláshoz szükséges struktúra
```

```
typedef struct
{
    ButtonState_TypeDef state;    // Állapot
    bool updated;                // Volt-e állapotváltozás?
} Button_TypeDef;

// Gombok paramétereit tároló struktúra
typedef struct
{
    Button_TypeDef up;            // Fel
    Button_TypeDef down;          // Le
    Button_TypeDef middle;        // Közép
    Button_TypeDef left;          // Balra
    Button_TypeDef right;         // Jobbra
} Buttons_TypeDef;

// Gombok állapotának lekérdezése
void Buttons_GetStates(Buttons_TypeDef * const buttons);

#endif /* INC_BUTTONS_H_ */
```

buttons.c

Ez a fájl tartalmazza a megvalósítását a buttons.h által publikált Buttons_GetStates() függvénynek, ami nem csinál mást csak a lokálisan karbantartott Buttons nevű és Buttons_TypeDef típusú adatstruktúrát bemásolja a paraméterben kapott ugyanilyen típusú struktúrába, valamint frissíti a gomb állapotváltozását figyelő updated nevű kapcsolót (flag).

A gombok állapotát a gombokhoz rendelt megszakítások figyelik, és frissítik a Buttons változó tartalmát. A gombok sajnos peregnek és a fejlesztőkártya nem tartalmaz hardveres pergesmentesítést. A perges fogalmával a 7. tananyagrészen már megismerkedhettünk, ismétlésképpen arról van szó, hogy 1 gombnyomást többnek érzékel a vezérlő, mert a fém kontaktusok a gomb belsejében nem egy alkalommal érintkeznek a nyomás vagy elengedés pillanatában. A kód egyfajta pergesmentesítést is tartalmaz az által, hogy az összes perges generál egy megszakítást, és a legvégül lefutó HAL_GPIO_EXTI_Callback() mindig a helyes állapotot fogja rögzíteni.

Applikáció

Az applikáció, azaz az ébresztőóra komponensei az alarm_clock.h és alarm_clock.c fájlokban találhatóak.

alarm_clock.h

Ez a fájl csak egyetlen sor kódot tartalmaz, ami nem más, mint az ébresztőóra szoftvermodul kifelé látszó egyetlen AlarmClock_MainFunction() függvényének a publikálása.

Ez a függvény hajtja az állapotgépet, ezért periodikusan meg kell majd hívni ahhoz, hogy az applikáció működni tudjon.

alarm_clock.c

Tartalmazza az AlarmClock_MainFunction() megvalósítását, valamint egyéb fájl szintű láthatósággal rendelkező (static) függvényeket, illetve magában foglalja az adatstruktúrát is.

Adatstruktúra

Az ébresztőóra állapotleírásánál meghatározásra kerültek a lehetséges státuszok. A fő állapotokat a MainState_TypeDef, az alállapotokat pedig a SubState_TypeDef felsorolások nevesítik. A kétféle, de összetartozó adatot a State_TypeDef fogja össze, amiből egy példány létezik, és a State nevet viseli. A TimeObject_TypeDef struktúra a kijelzőn megjelenített információt írja le olyan módon, hogy megadja a kezdőkoordinátákat és a kiválasztott adat típusát.

```
// Fő állapotok
typedef enum
{
    NORMAL_OPERATION = 0,
    ALARM_IN_PROGRESS
} MainState_TypeDef;

// Alállapotok
typedef enum
{
    SHOW_DATE_TIME = 0,
    SHOW_ALARM_PRESET_TIME,
    SET_DATE_TIME,
    SET_ALARM
} SubState_TypeDef;

// Állapotokat tartalmazó struktúra
typedef struct
{
    volatile MainState_TypeDef MainState;
    SubState_TypeDef SubState;
```

```
} State_TypeDef;

// Választható adattípusok felsorolása
typedef enum
{
    SELECTED_YEAR = 0,
    SELECTED_MONTH,
    SELECTED_DAY,
    SELECTED_HOUR,
    SELECTED_MINUTE,
    SELECTED_SECOND,
    SELECTED_ALARM_STATE
} Selection_TypeDef;

// Struktúra a kijelzőn megjelenítésre kerülő információkhoz
typedef struct
{
    Selection_TypeDef Selection;
    uint8_t x1;
    uint8_t y1;
} TimeObject_TypeDef;

// Állapotokat tartalmazó struktúra típusú változó
static State_TypeDef State;
```

Állapotgép

Az állapotgép megvalósításában több függvény is szerepet játszik, viszont kettőnek kitüntetett szerepe van. Ezek az úgynevezett belépési pontjai az applikáció szoftvermodulnak, ami annyit tesz, hogy ezeket a függvényeket lehet meghívni a részegységen kívülről.

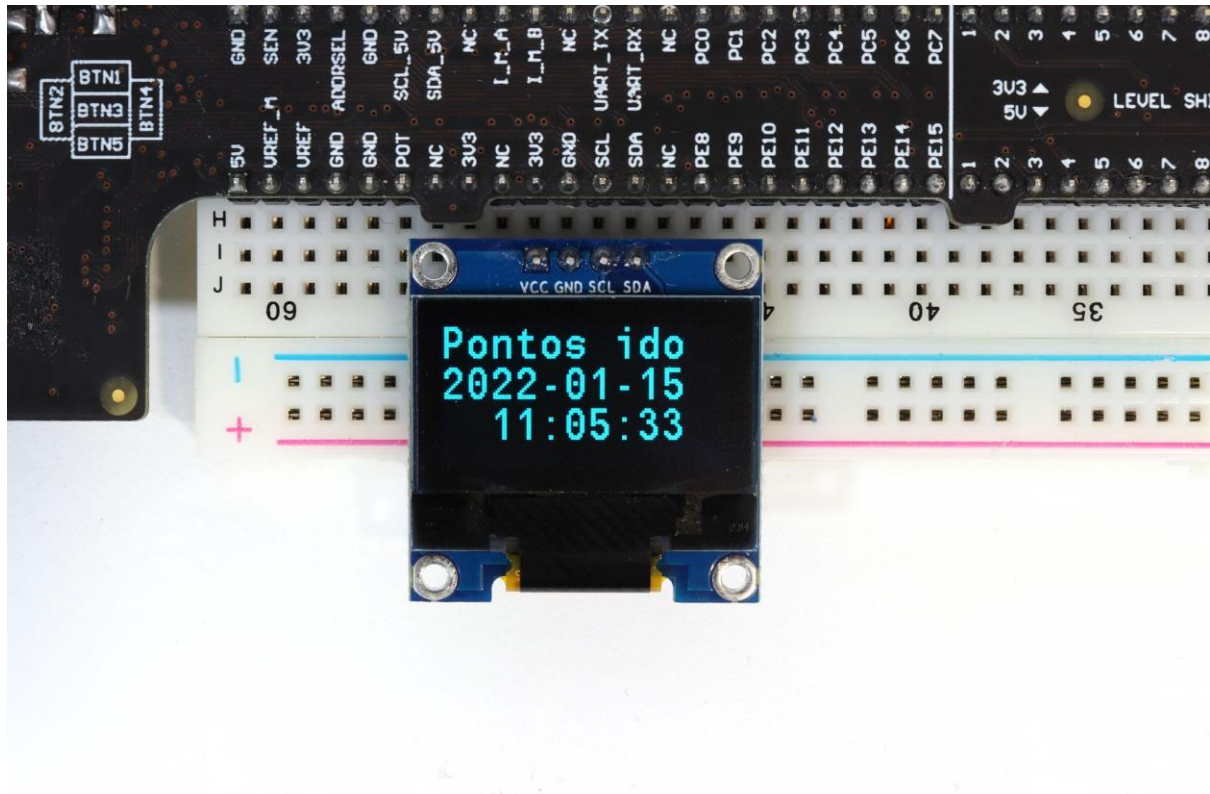
Az AlarmClock_MainFunction() hajtja az állapotgépet. Mit takar ez? Először is mindig lekérdezi a gombok állapotát és a dátumot, az időt, illetve az ébresztés idejét. Ezen információk alapján, és az állapotváltozó (State) megvizsgálásával az egyes State-eket kezelő függvények közül meghívja a megfelelőt, valamint amennyiben ez szükséges állapotot is vált.

A HAL_RTC_AlarmAEventCallback() egy olyan függvény, ami egy speciális kontextusból kerül meghívásra: megszakításból. Ha a beállított ébresztés éppen elkezdődik, akkor a vezérlés ide fog

kerülni attól függetlenül, hogy éppen hol volt előtte, és itt csak a fő állapotot állítjuk át ébresztésre (ALARM_IN_PROGRESS).

1.1.1.1.1 Óra és dátum kijelzése

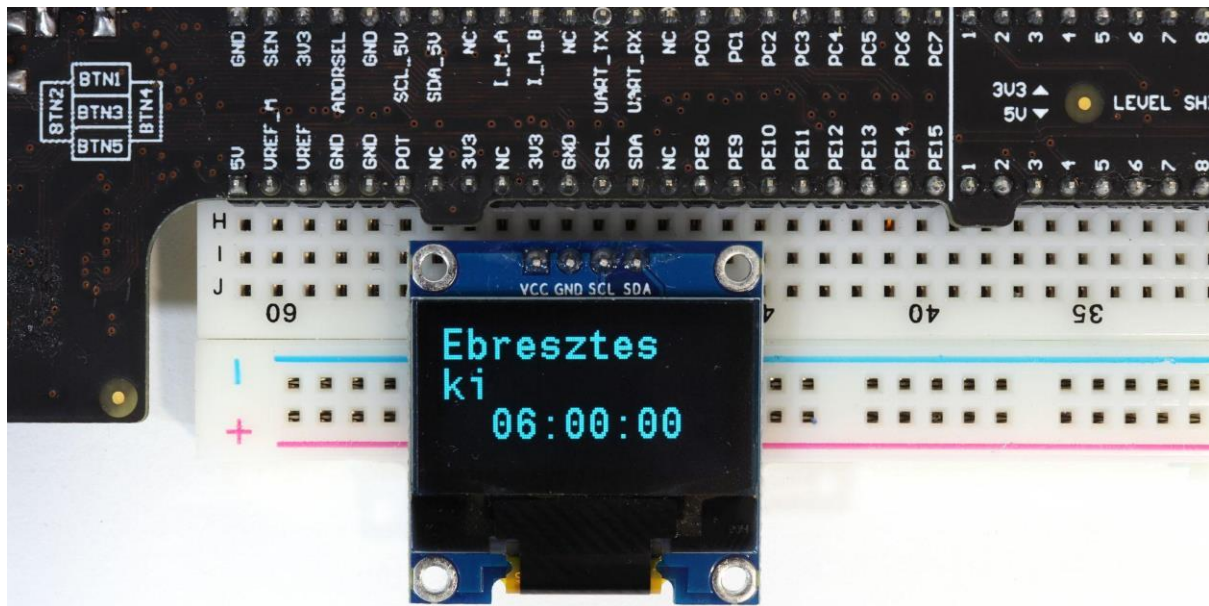
Az "óra és dátum kijelzés" üzemállapotot kezelő függvény neve a ShowDateTime(), ami paraméterként kapja meg az időt és a dátumot. Ezeket a kapott információkat kiírja az OLED kijelzőre az alábbi módon:



13. ábra: Pontos idő kijelzése

Ébresztés kijelzése

Az "ébresztés kijelzés" üzemállapotot kezelő függvény neve a ShowAlarmPresetTime(), ami paraméterként kapja meg a beállított ébresztés idejét és állapotát. Ezeket a kapott információkat kiírja az OLED kijelzőre az alábbi módon:



14. ábra: Ébresztési idő megjelenítése

Óra és dátum beállítása

Ezt az állapotot a `SetDateTime()` nevű függvény kezeli. Paraméterként megkapja a jelenleg beállított időt és dátumot, illetve a gombok állapotát. Amikor legelőször hívódik meg ez a függvény az "óra és dátum beállítása" üzemállapotban, akkor le kell menteni az időt és a dátumot ideiglenes változóba (`date_to_set` és `time_to_set`). Hogy miért? Azért, mert ezeken a változókon dolgozunk a gombok nyomogatásával és csak az állapotváltás során (T2) kell beírni ezeket az értékeket a mikrokontroller megfelelő regisztereibe a `HAL_RTC_SetDate()` és a `HAL_RTC_SetTime()` függvények segítségével. A beállítást segitendő létrehozunk egy tömböt `time_objects` névvel, amiben leírjuk, hogy adott információnak a kijelzőn hova kell kerülnie. A tárolt objektumok `TimeObject_TypeDef` típusúak és 3 dolgot tartalmaznak: az idő mely része van épp kijelölve (év, hónap, nap, stb.), kezdő x koordináta és kezdő y koordináta.

A balra és jobbra gombok nyomogatásával a kijelölést tudjuk módosítani, aminek a hatására a kód a `selection_idx` nevű változó értékét növeli vagy csökkenti, és segítségével végig tud lépkedni a már előbb említett `time_objects` tömb egyes elemein. A kijelölés a kijelzőn egy villogó kurzor formájában jelenik meg az adott érték alatt.

A fel és le gombok hatására az éppen kijelölt értéket lehet csökkenteni vagy növelni. Az átfordulásokat és az értékhatárokat az `UpdateSelection()` segédfüggvény ellenőrzi. Tegyük fel, hogy éppen szökőév van, a beállított hónap február, és éppen a napok vannak kijelölve, valamint az érték 28. Ha nyomunk egy felfelé gombot, akkor az `UpdateSelection()` engedni fogja a 29-et beállítani, és ha még egyszer felfelét nyomunk, akkor pedig az érték átfordul 29-ről 1-re.

Középső gomb megnyomására kilép a rendszer az "óra és dátum beállítása" állapotból, végrehajtva a T2-höz tartozó tevékenységeket.

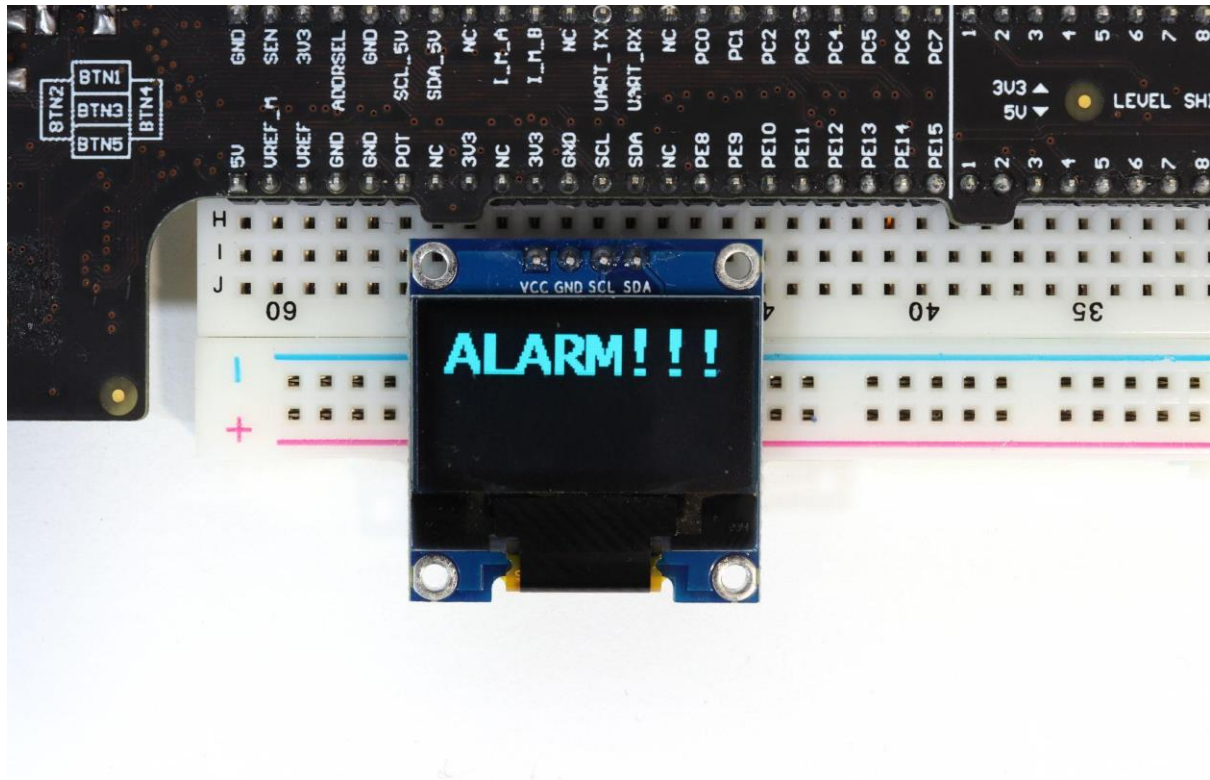
Ébresztés beállítása

Az ébresztés beállítását a SetAlarm() függvény végzi. A megvalósítása nagyon hasonlít az előzőekben tárgyalt SetDateTime() implementációjára, csak az információ más, amin dolgozni kell.

Ébresztés

Az ébresztésért az Alarm() nevű függvény felel. A kijelzőre kirajzol egy nagy "ALARM!!!" feliratot, amit másodpercenként kétszer villogtat. Egyszer világos háttér mellett sötéttel írja ki a feliratot, utána pedig fordítva.

A középső gomb megnyomására ki tudunk lépni az ébresztésből.



15. ábra: Ébresztés

main.c

Ez a fájl tartalmazza a projekt belépési pontjának számító main() függvényt, aminek a feladata a modulok inicializálása, és a későbbiekben majd az operációs rendszer elindítása lesz. Az ébresztőóra állapotgépezetnek a periodikus hívásáról egy egyszerű végtelen ciklus gondoskodik az alább látható módon:

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    // Ébresztőóra állapotgépe
```

```
AlarmClock_MainFunction();  
  
/* USER CODE END WHILE */  
  
/* USER CODE BEGIN 3 */  
  
}
```

Ez a megvalósítás csak olyan egyszerűbb rendszerekhez lehet megfelelő választás, ahol kevés feladatot kell végrehajtani, és nincsenek szigorú időzíítési feltételek. Több, és bonyolultabb feladathoz már operációs rendszer használata javasolt, amire a későbbi tananyagrészekben láthatunk majd példákat.

ÖSSZEFOGLALÁS

Ebben a részben az RTC perifériával ismerkedhettünk meg, először nagy általánosságban, utána pedig STM32F4 specifikusan. Megismertük a HAL függvénykönyvtár ide vonatkozó leggyakrabban használt függvényeit is, átnéztük a fejlesztőpanel RTC-vel kapcsolatos hardverkomponenseit, legvégül pedig áttekintettük egy egyszerű ébresztőóra megtervezését, és végighaladtunk a megvalósításának a részletein.